real-sw-p8-noshared

An AgentMPI run analysis

Abstract

Eight real Cursor subagents, one per module, built the `minidb` SQL engine with the shared interface window withheld: no rank could read the interface its neighbours published. Against the otherwise identical baseline `real-sw-p8-full`, the ablation cost 19.9% fewer prompt tokens in the implementation phase and bought 40.8% more completion tokens (40,775 → 57,393), 37.8% more delivered source (1,954 → 2,693 lines) and 4.7× the defensive-adaptation constructs (7 → 33; 3.58 → 12.25 per kloc), while acceptance moved by exactly one case out of sixty (60/60 → 59/60). Ninety per cent of the extra output is in `parser.py` and `engine.py`, the two modules whose withheld dependencies the broadcast specification does not pin, and the agents' own returned `concerns` name the mechanism: not failure but adaptation — `engine.py` routes 47 attribute-name guesses through one tolerant accessor rather than calling a published interface. The single most important reading is that withholding a shared interface does not show up as a broken build; it shows up as code that introspects its collaborators at runtime, as a 1.98 → 3.24 jump in the straggler-to-mean ratio at the integration barrier that lifts coordination from 41.7% of the pool's rank-seconds to 56.4%, and as $0.23 of extra completion tokens that buy nothing. Cleanest of all, the interface-publication phase still ran with the window closed: eight agent calls and 8,735 completion tokens spent on interfaces that only their own authors could read. The run also carries four harness defects and one AgentMPI accounting bug, all documented below.

1 What this run is

property	value
run	`real-sw-p8-noshared`
experiment	`software`
campaign label	`real-sw`
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 45
events	473
wall time	528.59 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	3.37×	total busy time over wall time
parallel efficiency	42.1%	achieved parallelism per rank
idle fraction	57.9%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	17.8%	time with exactly one rank busy
load imbalance	2.28×	slowest rank's busy time over the mean
coordination share	56.4%	rank-seconds blocked in collectives, of those available
coordination in flight	89.3%	wall clock with any rank inside a collective
rank reattachments	45	fresh agent processes that took over a rank role
agent calls	32	invocations that reached a model
agent latency p50 / p95	38.76 / 171.68 s	per invocation
messages	81	61 eager, 20 rendezvous
tokens sent	155,792	97,526 deferred under rendezvous
tokens in / out	94,239 / 72,935	prompt and completion
cost	$1.3767	0.0189 $/kTok out

3 What the analysis notices

- [critical] At least one rank waited 5 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- [warning] `neighbor_allgather/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [warning] `neighbor_alltoall/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [note] Collectives account for 56.4% of the run's rank-seconds, so more of the pool's time went to coordination than to work.
- [note] Load imbalance is 2.28×: the slowest rank was busy far longer than the mean.
- [ok] 45 rank reattachment(s) occurred, up to incarnation 7: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 10 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [warning] 3 of 10 collective(s) disagree with the cost model on *round* count while agreeing on messages: `barrier/central` recorded 0 against 2, `barrier/central` recorded 0 against 2, `barrier/central` recorded 0 against 2. Rounds are the critical-path term, so a disagreement here changes predicted latency without changing predicted traffic.

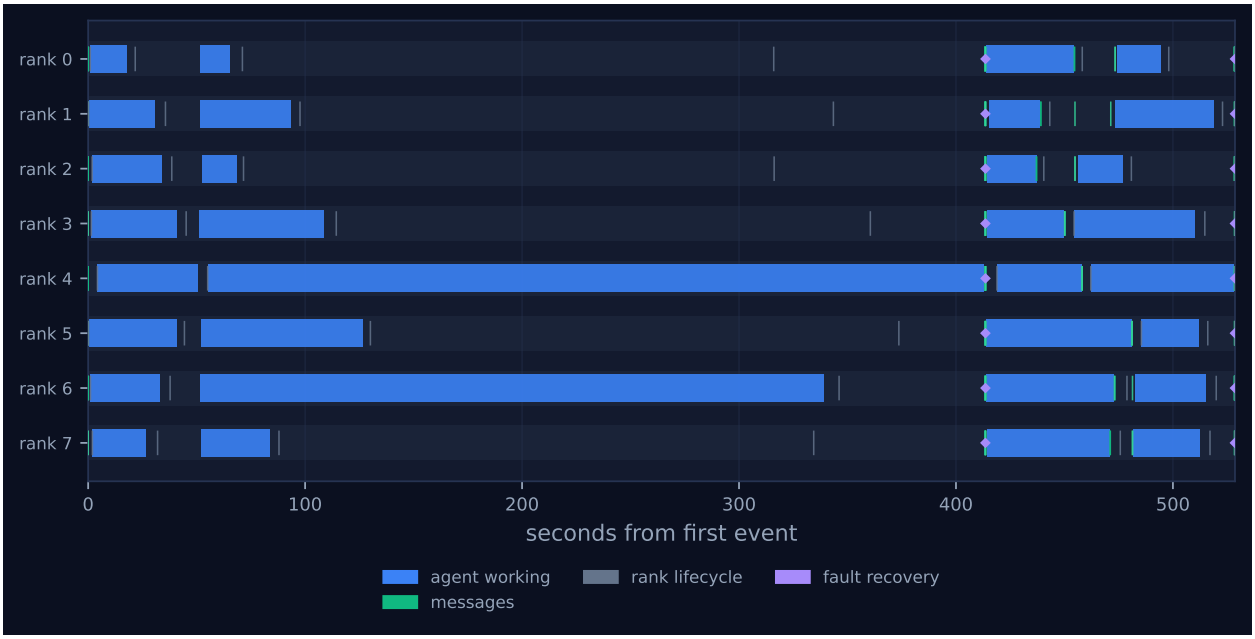

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

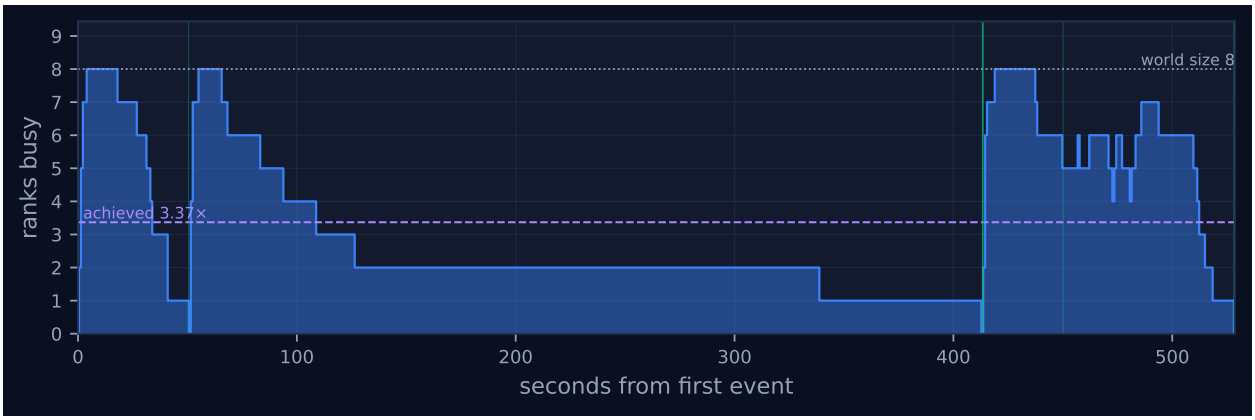

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

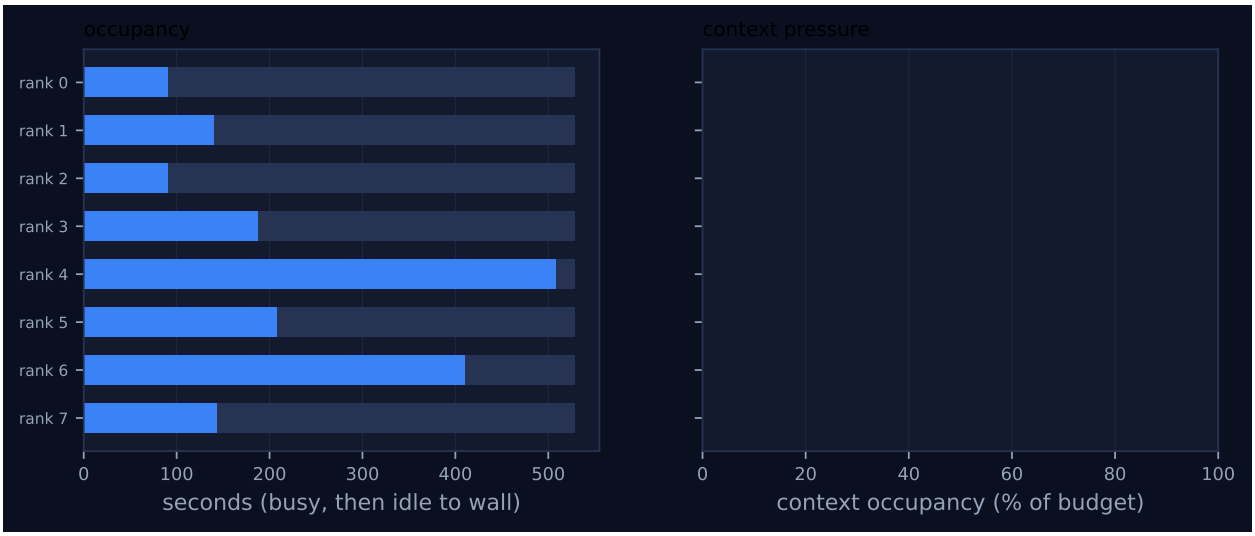

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	90.99	17.2	4	4	19	10	18,120	0.0	finalized
1	141.11	26.7	4	4	6	9	6,832	0.0	finalized
2	91.20	17.3	4	4	11	11	14,986	0.0	finalized
3	187.42	35.5	4	4	6	9	10,102	0.0	finalized
4	508.44	96.2	4	4	16	13	56,427	0.0	finalized
5	208.54	39.5	4	4	6	9	18,330	0.0	finalized
6	410.33	77.6	4	4	11	11	26,973	0.0	finalized
7	143.33	27.1	4	4	6	9	4,022	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

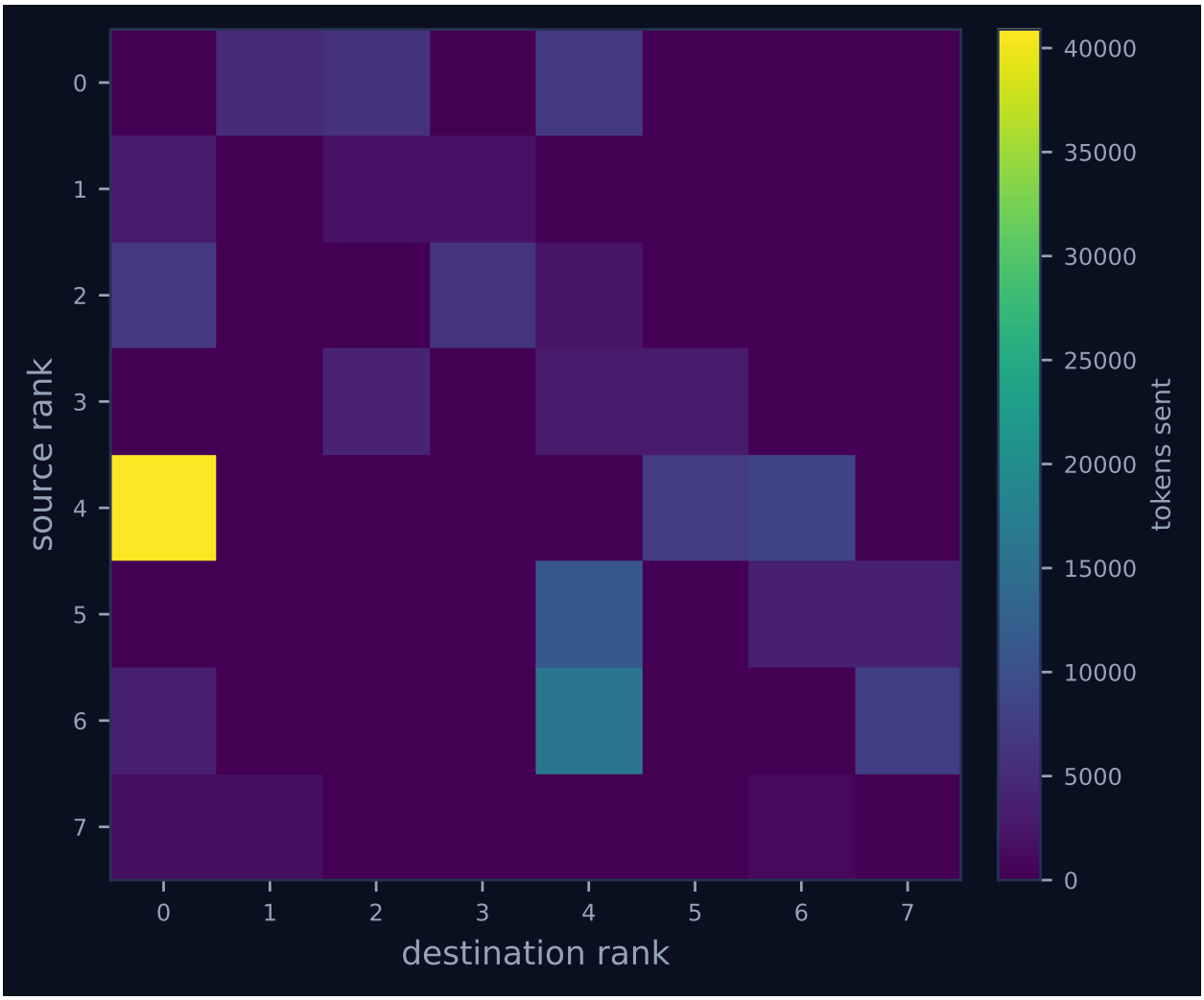

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

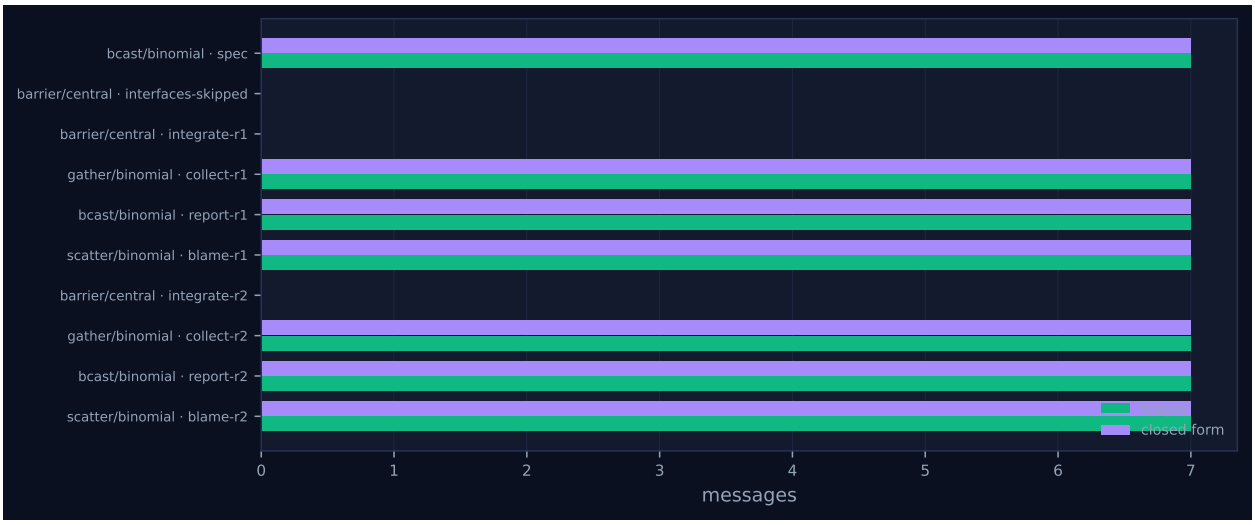

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	mo
bcast	binomial	spec	8	8	3	3	7	7	10,682	0.02	0.02	✓
barrier	central	interfaces-skipped	8	8	0	2	0	0	0	32.70	0.20	✓
barrier	central	integrate-r1	8	8	0	2	0	0	0	347.64	0.22	✓
gather	binomial	collect-r1	8	8	3	3	7	7	41,363	0.26	0.05	✓
bcast	binomial	report-r1	8	8	3	3	7	7	5,362	0.18	0.08	✓
scatter	binomial	blame-r1	8	8	3	3	7	7	5,904	0.01	0.08	✓
neighbor_allgather	?	review-r1	8	8	0	–	16	–	0	0.03	0.02	-
neighbor_alltoall	?	reviewback-r1	8	8	0	–	16	–	0	32.23	31.23	-
barrier	central	integrate-r2	8	8	0	2	0	0	0	50.96	0.18	✓
gather	binomial	collect-r2	8	8	3	3	7	7	41,537	0.25	0.11	✓
bcast	binomial	report-r2	8	8	3	3	7	7	5,362	0.24	0.09	✓
scatter	binomial	blame-r2	8	8	3	3	7	7	5,904	0.00	0.08	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has three blocks separated by two long silences, and every interesting property of this run lives in the silences.

The first block runs from $t = 0$ to $t = 50.5\,\mathrm{s}$ with all eight lanes solid: each rank is publishing the interface for the module it owns. This block is the run's first oddity. The window was never created — the fabric's `windows` table has zero rows and the `win.*` event kinds are absent from the vocabulary — yet the publication step still happened, because the harness gates the window on `cfg.shared_interfaces` but not the agent call that fills it. Those eight calls consumed identical prompt tokens to the baseline's (13,746 in both, since the interface prompt does not depend on the window), returned 8,735 completion tokens, and cost 276.6 s of summed agent latency. Their

only surviving consumer is the author itself, through the "interface you published" block of its own implementation prompt. Rank 4 finished last at 50.5 s and the substituted `interfaces-skipped` barrier released, having made the earliest arrival wait 32.70 s.

The second block, 50.7 s to 413.2 s, is the round-1 implementation, and it drains as a staircase rather than a block: `errors` at 65.3 s, `nodes` at 68.4, `api` at 83.4, `tokens` at 93.6, `functions` at 108.5, `planner` at 126.6 — and then nothing until `engine` at 339.0 s and `parser` at 412.7 s. Figure 2 shows the consequence directly: the achieved parallelism line sits at 3.37× against a world size of 8, and from roughly $t = 130$ s the plot is pinned at one or two. Computed from the claim/complete intervals in the log, this run spent 59.2% of its wall time with at most two ranks busy and 17.7% with exactly one, against 29.1% and 8.5% in the baseline; the longest stretch with no more than one rank busy is 75.0 s, from 339.0 s to 414.0 s, when rank 4 is alone. That is the whole explanation of the two flagged notes. The 2.28× load imbalance is rank 4 at 508.4 s busy (96.2% occupancy) against a mean of 222.7, and the 56.4% coordination share is seven ranks parked in `integrate-r1` while one agent call finishes. That one barrier accounts for 2,006.7 of the run's 2,383.2 coordination rank-seconds — 84.2% of all coordination, and 47.5% of the entire 4,228.7 rank-second budget — against 924.8 in the baseline. Neither number is protocol overhead. The `integrate-r1` barrier sent zero messages; it is pure schedule skew, and the skew is what the ablation produced.

Why those two ranks and not others is the load-bearing observation. Rank 4 owns `parser.py`, whose declared dependencies are `tokens`, `nodes` and `errors`; rank 6 owns `engine.py`, which depends on `planner`, `functions` and `errors`. They are the two modules with a non-trivial withheld dependency set, their round-1 agent calls took 362.6 s and 288.6 s against a baseline 172.6 s and 233.6 s, and they returned 7,676 and 7,995 completion tokens against 3,876 and 4,184. By contrast `errors.py` (no dependencies) came back byte-identical to the baseline's, and `nodes.py`, `tokens.py` and `functions.py` — no dependencies, or only the one-class `errors` module — came back within 5–15% of it. The straggler is not a random draw; it is the rank with the most missing information.

The third block, 413.2 s to 528.6 s, is the integration and repair round, and it is dense. Figure 1 shows the collective machinery collapsing into a fraction of a second: the `collect-r1` gather completes in 0.26 s, `report-r1` in 0.18 and `blame-r1` in 0.01, and the eight violet recovery diamonds at 413.6 s are the first `agree` vote. Then the review calls fan out, and the green message ticks scattered across 437–481 s are the review-return exchange, whose 31.23 s skew is the largest in the collective table — a reviewer handed 4,968 prompt tokens takes longer than one handed 839, so the neighbourhood exchange serialises on the slowest reviewer in each neighbourhood. The round-2 implementations then run, and rank 4 is again last at 528.0 s.

Figure 4 is the clearest single picture of what the ablation moved. One cell dominates: rank 4 to rank 0 carries 40,897 tokens in two messages, 26.3% of all traffic in 2 of 81 messages. That is the binomial gather's penultimate hop, and rank 4 sits on it while also owning the file that doubled — so the module that bloated is the module whose bloat travels furthest. The same edge carries 26,645 tokens in the baseline, so it grew 53.5%. Across both rounds the gather payload rose from 60,580 to 82,900 tokens (+36.8%), which is the source-code growth measured on the wire. Total tokens sent rose 17.1% on 48 *fewer* messages, taking the mean payload from 1,032 to 1,923 tokens.

Two things are missing from the picture and both matter. There is no amber or orange anywhere: the viewer's legend for this run reads `agent working`, `messages`, `rank lifecycle`, `fault recovery` and nothing else, where the baseline's trace carries 116 `win.*` events (108 of them in the analyser's `rma` role). The one-sided lane is empty because the window was never opened, and that is the ablation made visible. Second, the context panel of Figure 3 is zero for all eight ranks, and the per-rank table reports 0.0% occupancy throughout. That is not health: the harness passes `admit=False` on

every message and every collective, so every one of the 81 arrivals is recorded `admitted: false` with `admitted_tokens: 0` and the context accountant never sees anything. The column carries no information for this run.

7 Interpretation

7.1 What is true by construction

Four differences from the baseline follow mechanically from the flag and should not be read as results. There are no window operations at all, so the step that renegotiates an interface under an exclusive lock never ran and the contention report in the result file is empty. The fence was replaced by an ordinary `barrier`, which changes the barrier count from 32 to 24 and the algorithm mix from sixteen `dissemination` plus sixteen `central` to twenty-four `central`. That substitution accounts for the entire message-count difference: the baseline's two dissemination fences logged 24 messages each, 48 in total, and $129 - 81 = 48$ exactly. Both runs ran the fixed two rounds and neither agreed green.

The fourth deserves its own sentence, because it is the cleanest single statement of what the ablation costs. Closing the window did not switch off the phase that fills it: all eight ranks still published an interface, on prompts identical to the baseline's down to the token (13,746 in both), returning 8,735 completion tokens over 276.6 s of summed agent latency — and with no window to put them in, each interface reached exactly one reader, its own author, through the "interface you published" block of that rank's implementation prompt. Eight agent calls were paid for at full price to tell eight agents something each of them already knew. That is not a subtle statistical effect that needs a baseline to see; it is a whole phase of the protocol running with its output disconnected, and it is visible in the trace as eight `agent.call` events with no `win.put` anywhere after them.

7.2 What emerged

The ablation's cost is almost entirely in completion tokens and in the source they encode, and the accounting closes exactly. Implementation prompts shrank by 13,529 tokens (67,895 → 54,366, −19.9%), which is the withheld interface text. Implementation output grew by 16,618 tokens (40,775 → 57,393, +40.8%). Of that growth, 14,988 tokens — 90.2% — is four numbers: `parser.py` at +3,800 and +3,559 tokens in rounds 1 and 2, and `engine.py` at +3,811 and +3,818. At the calibrated \$3/\$15 per million in and out, the run's \$1.3767 against the baseline's \$1.1444 is +\$0.2323, and $18{,}064 \times 15\text{e}{-6} - 12{,}879 \times 3\text{e}{-6} = \$0.2323$ reproduces it to the cent. So the ablation is a net loss in money as well as in quality: the prompt tokens it saves are worth a fifth of the completion tokens it provokes.

On disk the same growth reads as 1,954 → 2,693 lines at round 1 (+37.8%), concentrated exactly where the token growth is: `parser.py` 362 → 711 lines and `engine.py` 429 → 760, against `errors.py` and `api.py` byte-identical to the baseline's and `tokens.py`, `nodes.py` and `functions.py` within 15%.

The acceptance result is the least interesting number in the run and needs recovering before it can be quoted at all. The harness recorded `importable: false, n_total: 0` for both rounds, which is false; see §7.4. Re-scored against the current suite (`reeval_software.py`, suite `0eaa6b695b83`) the delivered tree passes 59 of 60 against the baseline's 60 of 60, the single failure

being `engine/empty_table_select` ("QueryError: the select list is empty"). One case in sixty is the near-null result the project already knows about, and it is the wrong place to look.

The right place is the shape of the code, and there the ablation is unambiguous. The re-scored defensiveness count — three-argument `getattr`, `hasattr`, and handlers for `AttributeError`, `KeyError`, `TypeError` or `IndexError` — is 33 constructs at 12.25 per kloc against the baseline's 7 at 3.58, a 3.42× increase in density, and its per-file distribution is the dependency graph: `planner.py` 16, `parser.py` 11, `engine.py` 4, `functions.py` 2, and zero in the four modules that either have no dependencies or depend only on `errors`. The baseline's seven are three in `planner.py`, three in `engine.py` and one in `functions.py`, with `parser.py` at zero. The metric understates the gap, because this population factored its defence: `engine.py` defines one accessor `_field(obj, names, default)` that walks a tuple of candidate attribute names, and calls it from 47 sites, each site a different guess at what `planner.Plan` or a `nodes` dataclass actually named a field. Counting reflective access sites textually gives roughly 73 in this tree (47 `_field` calls, 16 `getattr` in `planner.py`, 7 plus `inspect.signature` and `dataclasses.fields` reflection in `parser.py`) against 5 in the baseline, whose `parser.py` contains no reflective access of any kind.

What makes this more than a code-style observation is that the population diagnosed it itself, in two independent channels. In their returned `concerns`, all six ranks owning a module with declared dependencies state that the interfaces were unavailable and describe what they did instead. The `parser` owner: "…so parser.py cannot know the exact field names of the AST dataclasses or the exact token `kind` strings. Rather than invent signatures, it adapts… each node is constructed by matching our logical values onto the class's real field names (via `dataclasses.fields` or `inspect.signature`…)." The `engine` owner: "…the field names of planner.Plan and of the nodes.py AST classes are unknown. The engine therefore reads the plan and every expression node through a tolerant accessor… a published field list for planner.py and nodes.py would remove the guesswork." And in the review round, peers flagged the adaptation as a contract violation rather than as prudence: "`_field`/`_source_entry`/`_descending` guess attribute names… that the Plan interface does not publish; whichever attribute happens to exist first wins, so a rename in planner.py silently changes behaviour." The defensive coupling claim therefore does not rest on a metric we chose; it rests on a metric we chose agreeing with what both the authors and their reviewers said in prose.

The parallelism cost is real but subtler than it first looks, because it is not a cost in total work. Summed round-1 implementation latency is 894.3 s here against 944.3 s in the baseline — *lower* — while the straggler-to-mean ratio is 3.24 against 1.98. The ablation did not make the population do more agent-seconds of work in round 1; it made the distribution of that work far more skewed, and a barrier converts latency variance into idle rank-seconds. Coordination rose from 41.7% of the baseline's rank-second budget to 56.4% of this run's, in absolute terms +717.0 rank-seconds, and that total decomposes exactly: the round-1 integration barrier contributes +1,081.9 on its own, partly offset by the two window fences the ablation deletes (−220.6) and by a round 2 with less to do (−271.2). Summing the two integration barriers gives 2,163.8 rank-seconds against 1,353.1, which is the same +59.9% that summing the harness's own per-rank `integrate_s` timings gives independently (2,167.0 against 1,357.2; the 3-second excess is the gather, broadcast and scatter that timing bundles in). Total busy time is 1,781.4 rank-seconds against 2,270.2 and the idle fraction is 57.9% against 43.2%; of this run's 2,447.1 idle rank-seconds, 2,383.2 — 97.4% — were spent blocked inside a completed collective, so essentially all of the idleness is barrier waiting rather than scheduling gaps. This is the general lesson the run contributes to the sweep: information asymmetry among ranks is a load-balancing problem, and any collective that synchronises the population pays for it at the slowest rank.

The bloat also degraded the review phase, which is a second-order effect worth naming because it compounds. The executed harness sliced each reviewed file at 9,000 characters. Because the ablated `parser.py` and `engine.py` are roughly twice the size, that window covered 35.3% and 32.9% of them against 69.9% and 62.5% in the baseline, and three of the eight reviewers spent a finding on the cut instead of on the code ("the excerpt ends mid-expression in `_descending`"; "the listing is truncated before `_run`, `_eval`, the aggregate evaluation and the negative LIMIT/OFFSET check"; "the listing is truncated inside `_make_select`"). Defensive bloat consumes the review budget that would have caught it.

7.3 The flagged findings

The two [warning] items are correct and are instrumentation gaps rather than protocol defects. At the commit that executed, `topology.py` emitted only `indegree`, `outdegree`, `wall_s` and `label` for the neighbourhood collectives; the event payloads in this trace contain no `messages_sent` or `tokens_sent` field at all, so the analyser's comparison of the collective's self-report against the 16 messages the fabric logged under `_ampi:nbr_ag` and `_ampi:nbr_a2a` has nothing to compare with and reports zero. The current source emits both fields, with a comment saying precisely why: "the entire claim of a neighbourhood collective is that it costs Θ(degree) rather than $\Theta(p)$, and a trace that records only the degree leaves that claim unverifiable from the log." Expected for a run of this age; already fixed.

Both [note] items are the ablation's signature rather than incidental. The 56.4% coordination share and the 2.28× imbalance are the same fact — one straggler per round — expressed twice, and both moved in the predicted direction from the baseline's 41.7% and 1.64×. The coordination note is now a discriminator rather than a description: it fires here, because more of the pool's rank-seconds went to blocking than to work, and does not fire on the baseline. It could not do that job before the metric was corrected, when both configurations read above 80% and the gap between them was compressed into six points.

The two [ok] items are unremarkable. Forty-five reattachments to incarnation 7 is the broker executor's normal behaviour — a worker process attaches, claims, completes, and the rank role with its mailbox and context account outlives it — and the difference from the baseline's 38 is a property of the driving pool, not of the run. Ten of ten checkable collectives matching their closed-form message count is the reassurance it appears to be; the baseline scored 11 of 11, the extra one being a window fence that does not exist here.

7.4 Defects

An AgentMPI accounting bug, since fixed. `tokens_deferred` used to be incremented on two different axes and summed into one field. In `src/agentmpi/comm.py` the send path added `blob.tokens` when the chosen transport was `RENDEZVOUS`, and the receive path added `row["tokens"]` whenever `admit` was false. This harness passes `admit=False` on every receive, so every rendezvous message was counted twice. The arithmetic is exact: the 20 rendezvous sends carry 97,526 tokens, all 81 arrivals carry 155,792, and $97{,}526 + 155{,}792 = 253{,}318$, which is the `tokens_deferred` value in this run's `job.finish` event and archived result file. The baseline reproduces it: $79{,}354 + 133{,}090 = 212{,}444$. The tell is visible without any arithmetic: the archived result reports more tokens deferred (253,318) than sent (155,792), which a subset of sent traffic cannot be. `rank.py` documented the field with a single definition ("tokens that a rendezvous transfer avoided

moving into context") that the two increment sites did not both implement — the receive-side one measures what an *admission* decision avoided, a different quantity that is worth having under its own name.

Both quantities now have one. `cost.summarise` derives `tokens_deferred` (send-side, rendezvous only, a subset of `tokens_sent` by construction) and `tokens_unadmitted` (receive-side, any transport, arrived and never admitted into context) from the log, and `job.totals()` reports both; for this run they are 97,526 and 155,792. The log-derived path in `analysis.py` was always right, which is why the 97,526 in the headline table above needed no correction. The reason the conflation survived is instructive: the old test asserted that the *receiver* had `tokens_deferred > 4000`, encoding the conflation as expected behaviour. `tests/test_analysis.py` now asserts across all 500 archived runs that deferred tokens never exceed sent tokens, which is a property of the definition rather than of any one run.

The blast radius was narrower than I first claimed and the correction sharpens the finding rather than weakening it. I wrote that the inflated value reached the transport table built at `scripts/make_tables.py:371`; it did not. That table is built from the transport microbench's own result file, `free-transport.json`, and that microbench admits its receives — which is why its eager rows correctly read zero deferred tokens at every size. No published number was affected. What was affected is every archived agent-experiment result: `results/software/*.json` and `results/translation/*.json` still carry the runtime's figure, 2.6× high in this run's case, and none of them carries `tokens_unadmitted` because they predate the split. Anyone reading those files directly, rather than re-deriving from the traces, is reading an inflated number.

The oracle the population saw was a phantom. The executed `run_acceptance` parsed the suite's report with `json.loads(out[out.rfind("{"):])`, which slices from the last brace in the output — inside a nested per-case object — and never parses; the fallback stored the last 2,000 characters as an import error. Both rounds therefore reported `importable: false` with `n_total: 0`, and the `blame` scatter delivered one synthetic `IMPORT` entry per rank. The trace shows this without reference to the source: the scatter payload digests are identical between rounds — `bc89dfebb450` to rank 4, `d5c25c881181` to rank 2, `bc32845d645c` to rank 1 in both `blame-r1` and `blame-r2` — so round 2 was handed byte-for-byte the same definition of done as round 1, and both `agree` votes returned `false` with all eight voters and nobody excluded. Re-scoring the delivered trees off-trace gives 59/60. Round 2 was consequently near-inert here: seven of the eight files are byte-identical to round 1 (only `engine.py` changed, 760 → 771 lines), against six of eight changed in the baseline. The phantom cannot explain that difference on its own, since the baseline received the same phantom; what distinguishes them is that the baseline's round 2 had a genuinely new input — renegotiated dependency interfaces read back through `win.get` — and this run's only new input was a misrouted peer review. Fixed in the current `parse_report`, which is a named function with a test.

The review return path was misrouted. The executed harness returned reviews over the same topology it had distributed code on, passing `topo` to `neighbor_alltoall` rather than its transpose. With `review_edges(8, fanout=2)` the destinations of rank i are $\{i{+}1, i{+}2\}$ and its sources $\{i{-}1, i{-}2\}$ — confirmed by the `topo.graph_create` payloads, e.g. rank 7 with destinations $[0, 1]$ and sources $[5, 6]$ — so rank i received critiques written by ranks $i{-}1$ and $i{-}2$ about the modules of ranks $i{-}2$, $i{-}3$ and $i{-}4$, a set that never contains i for $p = 8$. No author received a review of their own module, and two of them said so in their next report: the `errors` owner wrote

"The peer review reports no defect in errors.py; every finding concerns parser.py, planner.py or engine.py", and the `api` owner "Every review item in this round is against minidb/functions.py and minidb/parser.py". The phase cost 8 agent calls, 26,127 prompt tokens, 6,807 completion tokens and 355.6 s of summed latency. It was not a total loss — reviewers routinely wrote findings whose *fix* named a third module, and the `planner` owner acted on five of them, opening five round-2 concerns with "Review point taken" — but that is luck, not routing. The current source builds a transpose topology for the return path and filters findings by owning path.

The ablation prompt understated the condition, and it did not matter here. All sixteen implementation prompts in `runs/real-sw-p8-noshared/spool/` print "PUBLISHED INTERFACES OF YOUR DEPENDENCIES: (this module has no dependencies)". For `errors` and `nodes` that is true; for the other six it is false, and for `api`, whose entire job is composing the other modules, it is badly false. The condition should have said the interfaces were withheld, not that the dependencies did not exist. In this run the misstatement was inert: every one of the six affected ranks recovered the true condition and said so, because the broadcast `SPEC.md` names each module's permitted imports, and the `api` owner in particular wrote "No published interfaces were supplied for the sibling modules, so this file calls them exactly as the specification's module-layout table defines them: parser.parse(sql) → Select, planner.plan(select, tables) → Plan, engine.execute(plan, tables) → list[dict]". Its `api.py` uses four static relative imports and is byte-identical to the baseline's. The same defect did bite in the later vague-specification sibling, where the spec no longer names the boundaries: that run's `api.py` is 123 lines and resolves each stage with `getattr(module, name, None)`. So the defect is real and worth the fix it has since received, but it is not a reason to discount this run's numbers.

Provenance drift. The result file records commit `1f7bd01`, which already contains the fix to `parse_report` — yet the run exhibits exactly the failure that the fix removes, and its review-return call is the pre-fix form. `provenance()` calls `git rev-parse HEAD` when the result is written, not when the job is launched, and HEAD advanced during the campaign (the baseline step, nine minutes earlier, records `26ae006`). The commit field is therefore an upper bound on the age of the code that ran, not a description of it, which is worth naming given that its stated purpose is to let a number in the paper be traced back to the run that produced it.

8 Threats to this reading

The two runs' workers ran at different speeds, so every wall-clock comparison is contaminated. Implementation-phase throughput was 57,393/1,211.9 = 47.4 completion tokens per second here against 40,775/1,440.5 = 28.3 in the baseline, a 1.67× difference, and the fitted calibration agrees: $\alpha = 16.18$ s against 32.20 s, 54.99 tokens/s against 42.33. The direction is inconvenient in a useful way — this run's p50 agent latency is 41.0 s against the baseline's 59.5, i.e. *faster*, which is the opposite of what the mechanism predicts — and it means the +5.9% wall-time difference cannot be attributed to the ablation at all. Had this run's workers served at the baseline's rate, its extra 18,064 completion tokens alone would have added roughly 640 s of agent time. Conversely, the barrier-wait sums quoted in seconds (2,008.3 against 926.5) are partly exposed to whatever caused the rate difference, most likely queueing in the external worker pool. The dimensionless straggler-to-mean ratio (3.24 against 1.98) and the busy-count distribution are less exposed but not immune. Token counts, the delivered source, the defensiveness counts and the

re-scored acceptance result do not depend on service rate at all, and those are the numbers this analysis leans on.

One run per configuration, so nothing here is statistical. The acceptance delta is one case in sixty, comfortably inside what a single re-roll of eight agents could produce, and it should not be quoted as evidence of anything. The defensiveness and source-size deltas are large enough (4.7× and +37.8%) that a coin flip is an implausible explanation, but the reason to believe them is not their size: it is their localisation. The extra code and every reflective access site sit in exactly the modules whose dependencies were withheld *and* whose boundaries the specification leaves open, the modules with no dependencies came back byte-identical, and the agents' own prose names the mechanism. That is a mechanism argument standing in for a statistical one, and it is what n=1 permits.

The ablation was not clean: the broadcast specification is itself a shared interface. This run uses the signature-bearing `SPEC.md`, whose module-layout table already pins the signatures of `tokenize`, `parse`, `plan` and `execute` together with their return types, the `Token` field names and every AST class name. The agents said as much: the `tokens` owner wrote "the specification's module table gives tokens.py `errors` as its permitted import and states that QueryError lives there", and the `planner` owner "the code is written against the node names and fields the specification's module table states". What the spec does *not* pin is exactly where the defensiveness landed — the token `kind` vocabulary, the AST dataclass field names, and the shape of `Plan` — so this run measures the ablation only over the residual the spec leaves undetermined, and its 59/60 understates the cost. The `-vague-spec` sibling pair exists to remove the confound and shows a different signature: both trees reach 60/60, but the ablated one carries 3,564 lines against 2,203 and six surviving spellings across three naming axes against three. Read this run as a lower bound.

The own-interface echo leaks the very information the ablation withholds. The implementation prompt still contains the rank's own published interface, and a well-written interface names its dependencies' call shapes: `api`'s says it works "by calling parser.parse(sql), planner.plan(select, tables) and engine.execute(plan, tables) in that order". That is why the eight discarded interface calls were not wholly wasted and why `api.py` came out byte-identical to the baseline's. What this run ablates is the cross-rank half of the channel, not the channel.

Coordination share is not protocol overhead, and the context column is not health. At 56.4% the coordination share now is a proportion — rank-seconds blocked in completed collectives over the $8 \times 528.6 = 4{,}228.7$ available — but it still invites the wrong reading. The barriers that dominate it sent zero messages. It measures schedule skew, not the cost of the protocol, and on this run 84.2% of it is a single barrier waiting on a single agent call. The companion `coordination_span_share` of 89.3% answers a different question again — how much of the wall clock had coordination in flight anywhere — and the two should not be swapped for one another. Separately, the per-rank context column reads 0.0% for all eight ranks not because context was ample but because `admit=False` is passed everywhere, so `context_high_water` and `context_rejections` are structurally zero and the run says nothing about admission pressure.

The repair loop was never exercised, so this run says nothing about convergence. Because the oracle's report never parsed, neither round was a repair round in any meaningful sense:

the population was told twice that a build passing 59 of 60 cases had failed to import, and the second round's only real input was a misrouted review. Any reading of the round-1 to round-2 trajectory — including the observation that seven of eight files did not change — is a statement about the broken plumbing, not about whether agents converge under an interface ablation.

The defensiveness metric is a floor, and its denominator moves with the numerator. It counts syntactic three-argument `getattr`, `hasattr` and adaptive `except` handlers, so `engine.py` scores 4 while routing 47 attribute-name guesses through one helper, and `parser.py`'s `inspect.signature` and `dataclasses.fields` reflection is not counted at all. That biases the measured 3.42× downwards. In the other direction, the per-kloc normalisation divides by a line count that the ablation itself inflated, so the density ratio is more conservative than the absolute ratio (3.42× against 4.71×); both are reported above and the absolute count is the one that should be quoted with care, since a larger program has more places to be defensive.